SOLO LEVELING

// SYSTEM PORTAL & PROGRESSION PLATFORM

Complete Feature Analysis, Capability Breakdown, and Improvement Roadmap Report

Prepared By: System Analyst

Date: 13 Nov 2026

Scope: index.html, app.js, style.css (+ 270 animation frames)

Status: Single-file static web application

EXECUTIVE SUMMARY

The Solo Leveling System Portal is a gamified, browser-based real-life progression engine themed after the Solo Leveling universe. It combines an animated hero scroll narrative, an RPG player status window, timed daily quests with penalty mechanics, webcam-based Hunter Vision scanning, an AI chatbot advisor, a before/after transformation social feed, and an inventory/skill arsenal — all persisted via browser localStorage and styled with a neon cyberpunk aesthetic.

1. SYSTEM OVERVIEW

This is a client-side web application built with plain HTML5, CSS3, and vanilla JavaScript (no frameworks, no build step, no backend). It loads 270 pre-cached animation frames (ezgif-frame-001.jpg through ezgif-frame-270.jpg) to produce a smooth, scroll-synchronized hero narrative, then transitions into a dashboard of gamified productivity modules. Player state (level, stats, quests, reminders, transformations) is stored in the browser's localStorage, so progress persists across sessions.

Core Technology Stack

Layer	Technology	Purpose
Markup	index.html (HTML5)	Structure & semantics
Styling	style.css (CSS3)	Neon cyberpunk UI, responsive layout, animations
Logic	app.js (Vanilla JS)	All engines, state, audio, camera, AI chatbot
Media	270 JPG frames	Scroll-synced hero animation
Storage	localStorage API	Persistent player state (no server)
Audio	Web Audio API	Procedural sound effects (no files)
Camera	getUserMedia API	Webcam access for Hunter Vision
Fonts	Google Fonts (Orbitron, Rajdhani, Syne, JetBrains Mono)	Sci-fi typography

2. WHAT THE SYSTEM CAN DO — COMPLETE CAPABILITY BREAKDOWN

The application is organized into seven functional modules plus shared systems (audio, notifications, persistence). Below is a complete inventory of every capability.

MODULE 1 — HERO SCROLL ANIMATION (Story Track)

A 480vh scroll-track divided into 4 story chapters that synchronizes a 270-frame animation with the user's scroll position.

- Scroll-synced frame playback (270 frames at 30 FPS)
- Four chapters: The Awakening, Shadow Extraction, System: Arise, The Shadow Army
- Interactive playback deck: Play/Pause, Replay, 0.5x–2x speed, frame scrubber
- Keyboard shortcuts: Space, Left/Right arrows, R (reset), M (SFX), Home/End
- Jump pills to teleport to any chapter
- Chapter-based HUD (frame counter, % progress, chapter name)
- Ambient particle system and vignette/scanline overlay

MODULE 2 — PLAYER STATUS & RPG LEVELING ENGINE

A full RPG character window with level, stats, gold, fatigue, HP/MP/EXP gauges, and ability-point allocation.

- Player state: Level 18, Title 'The Shadow Monarch', Job 'Shadow Lord (Necromancer)', Rank S
- Attributes: STR 124, AGI 118, VIT 95, INT 165, PER 88
- Dynamic rank calculation (E → D → C → B → A → S/Monarch) based on level
- HP = 5000 + VIT×35 + Level×150; MP = 8000 + INT×40 + Level×200
- EXP bar with level-up triggers (+3 AP per level, maxExp grows ×1.3)
- Unallocated AP distribution into any stat (+1 per click)
- Penalty system: level decrease + fatigue increase on quest expiry
- localStorage persistence of all player state

MODULE 3 — DAILY QUESTS & TIMED PENALTY ZONE

A quest log with countdown timers, rewards, and automatic penalties for expired quests.

- 3 preset daily quests (Physical Conditioning, Deep Focus, Mana & Hydration)
- Countdown timer per quest (ticks every 1 second)
- Urgent warning when < 5 minutes remain
- Auto-expiry → Penalty Zone (level decrease + fatigue spike)

- Quest completion awards EXP, Gold, AP, and reduces fatigue
- Custom quest creator with rank-based XP/Gold/AP rewards
- Re-issuable custom quests (1–1440 minutes)

MODULE 4 — HUNTER VISION & CAMERA AI SCANNER

Webcam-based biometric/posture assessment plus a reminder/alarm scheduler.

- Start/stop webcam via getUserMedia (user-facing camera)
- Capture frame and render to canvas
- Simulated AI assessment report (posture integrity, fatigue, task verification)
- System recommendation output after scan
- Reminder scheduler with countdown and browser notifications
- Toast notifications when reminders fire
- Fallback path when camera is denied/blocked

MODULE 5 — SYSTEM AI CHATBOT ('THE ARCHITECT')

A rule-based knowledge-base chatbot that answers leveling, stat, and workout questions.

- Chat interface with user vs. system message bubbles
- Knowledge base with keyword matching (leveling, stats, penalty, workout, shadow)
- Quick-query chips for common questions
- Typing-delay simulation for responses
- Fallback calibration response for unknown queries

MODULE 6 — HUNTER PROGRESSION & TRANSFORMATION FEED

A before/after social feed where users post transformation updates with explicit consent.

- Post transformation updates (before photo + after photo + caption)
- FileReader DataURL storage of images in local state
- Explicit privacy permission checkbox before posting
- Like / 'Arise Respect' counter with toggle
- Sample showcase card (Sung Jin-Woo, 90 days of progress)
- User cards prepended to the feed

MODULE 7 — INVENTORY & SHADOW SKILLS ARSENAL

Dimensional storage slots and a Monarch skill list with levels and descriptions.

- 8 inventory slots (4 equipped, 4 empty): Knight Killer, Rasaka's Fang, HP Potion ×5, Dungeon Key
- Item rarity badges (Rank B, C, S)
- Skill list: Sprint (Passive), Stealth (Active), Ruler's Authority, Shadow Extraction 'Arise'
- Per-skill level and description
- Gold counter (45,800 G) displayed in header

3. SHARED SYSTEMS (CROSS-CUTTING FEATURES)

Audio Synthesizer (Web Audio API)

- Procedural sound effects with no audio files: level-up fanfare, quest-done chime, penalty buzzer, UI click
- SFX toggle (mute/unmute) with visual state and toast confirmation
- AudioContext resume on user interaction (handles browser autoplay policy)

Holographic Toast Notification Hub

- Fixed-position notification toasts (top-right)
- Type-styled toasts: info, level-up (gold), penalty (red)
- Auto-dismiss after 4.5 seconds with slide-out animation

Persistence (localStorage)

- Player state saved as 'solo_leveling_player_state' JSON
- Survives page refresh and browser restart
- Auto-saved on every state mutation

Browser Notifications

- Requests notification permission on load
- Desktop notifications fire when reminders expire

Responsive Design

- CSS grid collapses to single column on ≤1024px and ≤768px screens
- Hidden nav tabs and condensed cards on mobile

4. STRENGTHS & WHAT WORKS WELL

- Zero dependencies — runs entirely in the browser with no build step, npm install, or server required.
- Strong thematic consistency — the Solo Leveling aesthetic (neon cyan/purple, Orbitron font, holographic cards) is applied throughout.
- Rich interactivity — scroll-synced animation, scrubber, keyboard shortcuts, and jump pills make the hero track engaging.
- Gamified psychology — XP, levels, AP, rank, gold, fatigue, and penalty mechanics create real motivation hooks.
- Client-side persistence — localStorage means no database or backend is needed for basic progress tracking.
- Privacy-by-design — transformation photos require explicit consent and are stored locally, never uploaded.
- Graceful degradation — camera access is optional; the app remains fully usable if the webcam is denied.
- Extensible modular structure — each module is a self-contained block of code, easy to extend or replace.

5. IMPROVEMENT ROADMAP — MAKING IT MUCH BETTER

The following are concrete, prioritized enhancements to evolve this prototype into a polished production-grade gamified productivity platform.

P0 — Critical / Stability

- Add input validation and sanitization for chatbot and quest fields to prevent XSS (user text is currently inserted via innerHTML).
- Replace innerHTML with textContent or a sanitizer for all user-generated content (quest titles, captions, author names).
- Add error boundaries / try-catch around canvas drawing to prevent a single bad frame from freezing the render loop.
- Persist quest completion progress (e.g., push-up counters) so partially completed quests survive refresh.

P1 — Core Experience

- Connect quest progress to real inputs: add checkboxes/numeric inputs so users can log actual push-ups, sit-ups, squats, km run, and focus minutes.
- Replace the rule-based chatbot with an LLM integration (e.g., OpenAI / local model) for genuinely useful fitness and study advice.
- Add a dark/light theme toggle and CSS custom-property theming.
- Add sound on/off per-category (UI vs. level-up vs. penalty) and a volume slider.

P2 — Social & Progression Depth

- Add friend/guild system and shared leaderboards (requires a backend).
- Add skill trees with prerequisites and unlock conditions instead of flat AP buttons.
- Add streak tracking (consecutive days quests completed) and streak rewards.

- Add image compression/optimization before storing transformation photos in localStorage (to avoid quota limits).

P3 — Polish & Distribution

- Convert to a PWA (service worker + manifest) so it can be installed offline and run like a native app.
- Add a backend (Node.js/Express or Supabase) for cloud sync, multi-user accounts, and public transformation feeds.
- Add unit tests (Jest/Vitest) for the leveling, quest, and timer logic.
- Add accessibility improvements: ARIA labels, keyboard focus indicators, reduced-motion support.
- Optimize the 270-frame animation: lazy-load frames or convert to a video/WebM for faster initial load.

6. HOW TO RUN LOCALLY — LOCALHOST LINK

Because this is a static front-end app (no backend), you only need a simple HTTP server to serve the files. Opening index.html directly via file:// may block the webcam and some features due to browser security restrictions, so a localhost server is recommended.

Option A — Python (built-in, no install)

Open a terminal in the project folder and run:

```
python -m http.server 8000
```

Then open in your browser:

```
http://localhost:8000
```

Option B — Node.js (npx http-server)

Run:

```
npx http-server -p 8080
```

Then open:

```
http://localhost:8080
```

Option C — VS Code 'Live Server' extension

Install the 'Live Server' extension, right-click index.html, and choose 'Open with Live Server'. It typically starts at:

```
http://127.0.0.1:5500
```

NOTE: For full functionality (Hunter Vision camera scan), you must serve via http://localhost (not file://) and grant camera permission when prompted.

Solo Leveling is owned by Chugong & Redice Studio. This is a fan-made gamified productivity prototype. All rights reserved to their respective owners.